Solving Polynomial Systems of using Linear Elimination

Ilay Menahem, Michael Shalyt, Ido Kaminer

March 30, 2026

Abstract

There is a problem shared among many fields of science of solving polynomial systems of equations. The required solution is often a space of solutions, rather than a single solution. Applications include QFT, robotics, cryptography, optimization, chemistry, electrical engineering, and more. [we need other examples... best is to have concrete examples from history. If not then maybe say generally that in many areas of science and engineering a parametric solution can surprise us and help where our creativity is limited].

This paper presents a scalable pipeline for finding a space of solutions (of full dimension) to polynomial systems of equations. We start from numerical solutions, transitioning to symbolic solutions, and finally parametrize the solution space via structured exploration. We then also discuss other complementary methods and discuss what open questions still remain. We demonstrate these methods on concrete examples from the field of QFT.

Introduction

Solving polynomial systems of equations is a fundamental problem in mathematics and computer science, with applications in fields such as robotics, cryptography, optimization, QFT, chemistry, electrical engineering, economics and more. In QFT, polynomial systems arise in the context of generilizing and unifying theories (e.g. quantum gravity). In chemistry, for example, polynomial systems appear in the study and design of reaction chains. In robotics, for example, polynomial systems are used to model kinematics and motion planning.

In most of these use cases, the required solution is often a space of solutions, rather than a single solution. Specifically, in QFT the proposed theory must follow

gauge invariance, which leads to a space of solutions, and of these solutions one may want to find the one represents the simplest theory. In chemistry, reaction chains often have multiple valid pathways, leading to a solution space. In robotics, the solution space may represent all possible configurations of a robotic arm that satisfy certain constraints. one would want to work with the entire solution space to find optimal paths and configurations. This type of problem is universal across many fields of science and engineering. It calls for a comprehensive approach that can efficiently find and represent the solution space of polynomial systems.

Classic methods for solving polynomial systems of equations include Groebner bases [], resultants [], and homotopy continuation []. However, these methods can be computationally expensive and may not scale well with the size of the system. Other approaches, such as gradient [], and SMT solvers [] are focused on finding specific solutions rather than exploring the entire solution space.

In this paper, classify/group/review/discuss specially made approaches for finding the general ... in efficient ways relevant for. We also demonstrate the methods on concrete examples from QFT. Among those methods, we propose a pipeline that leverages numerical methods to find initial solutions, then refines these solutions into symbolic forms, and finally explores a space of solutions. This approach aims to combine the efficiency of numerical methods, and some insights we may have at how the symbolic solutions may look like, to provide a comprehensive understanding of a solution space of polynomial systems. We also present and analyze a complementary examples from QFT, showcasing the effectiveness of our approach in practical scenarios. We finally outline the open questions and future directions [more concrete] in this field.

Mathematical Background

Let $\mathbb{K}[\mathbf{x}]$ denote the polynomial ring in variables $\mathbf{x} = (x_1, \ldots, x_n)$ over a field $\mathbb{K}$, typically $\mathbb{F}_q$, $\mathbb{R}$ or $\mathbb{C}$. A polynomial system is defined by a map $F : \mathbb{K}^n \to \mathbb{K}^m$ with components $f_1, \ldots, f_m \in \mathbb{K}[\mathbf{x}]$. The locus of simultaneous solutions constitutes the *affine variety* $V(F) = \{\mathbf{a} \in \mathbb{K}^n \mid F(\mathbf{a}) = \mathbf{0}\}$.

In this paper, we will mostly work over the rational, real or complex numbers.

Linear elimination

even in the case of very large non linear polynomial systems of equations, it is possible to eliminate variables in a linear fashion, which can significantly reduce the size of

the system and the number of variables.

Algorithm 1 Linear elimination by substitution

Require: Polynomial system $P = \{p_1, \ldots, p_k\} \subseteq \mathbb{K}[x_1, \ldots, x_m]$

Ensure: Reduced system Q where no $q \in Q$ is linear in any remaining variable; and a substitution map Σ to recover eliminated variables

$Q \leftarrow P$
$V \leftarrow \{x_1, \ldots, x_m\}$
$\Sigma \leftarrow \emptyset$
$L \leftarrow \{(x', p') \mid x' \in V,\ p' \in Q,\ p' \text{ is linear in } x'\}$
while $L \neq \emptyset$ **do**
    Select any pair $(x, p) \in L$
    Solve $p = 0$ for x to obtain $x \leftarrow f(\hat{x})$
    $\Sigma \leftarrow \Sigma \cup \{x \mapsto f(\hat{x})\}$
    Substitute $x \leftarrow f(\hat{x})$ in Q
    $V \leftarrow V \setminus \{x\}$
    Simplify the system Q ▷ Optional, but recommended
    clear Q of zero equations
    $L \leftarrow \{(x', p') \mid x' \in V,\ p' \in Q,\ p' \text{ is linear in } x'\}$
end while
if $Q = \emptyset$ and $\Sigma \neq \emptyset$ **then**
    Validate: back-substitute Σ into each $p \in P$; each resulting polynomial must be identically zero (probabilistic zero test in the implementation)
end if
return (Q, Σ)

we have the freedom to choose which linear variable to eliminate at each step. good heuristics to choose which variable to eliminate include:

- **Lowest degree in the other equations:** eliminate the variable that appears with the lowest degree in the other equations first.
- **Lowest degree after substitution:** eliminate the variable that will lead to the lowest degree in the other equations after substitution.
- **Greatest or least number of occurrences:** eliminate the variable that appears in the greatest or least number of equations first.

It is also possible to use DFS or BFS to search on the order of elimination.

Validation of the solution

After obtaining a solution, we can validate it by substituting the solution back into the original system and checking if it satisfies all the equations.

To check whether the solution is of full dimension at certain points, we can use the following theorem:

Theorem 1 (Gatilov et al. [1]) *Given a continuously differentiable system $F \in C^1$ (possibly polynomial) with a solution point p, the dimension of the solution space in the neighborhood of p is $n - rank(J_F(p))$, where J_F is the Jacobian matrix of F at point p.*

System dataset

For linear elimination we need to be able to do the following operations efficiently

- is polynomial linear in a variable
- is polynomial zero
- linearly eliminate a variable from a polynomial

To prevent an exponential blowup in the size of the equations, we store the current polynomial system in unexpanded form. Each polynomial expression is represented as a DAG, so common subexpressions are shared and elimination steps do not force an immediate expansion of all dependent equations. The algorithm therefore works directly with graph-represented polynomials.

Once the system is stored in this form, linearity and vanishing can no longer be decided from the top-level syntax alone. Instead, the required operations are phrased in terms of evaluation of the polynomial represented by the DAG.

The techniques outlined here can be immediately extended to rational functions.

Evaluating a DAG-represented expression

Let G be a DAG representing a polynomial expression, and let α be an assignment of values to the variables occurring in G. To evaluate the represented expression, we process the nodes of G in topological order, so that every child of a node is evaluated before the node itself.

Algorithm 2 Evaluating a polynomial expression represented as a DAG

Require: DAG G with root r, and an assignment α of values to the variables of G
Ensure: The value of the polynomial represented by G at α

for each node v of G in topological order **do**
  if v is a constant node with value c **then**
    $\mathrm{val}(v) \leftarrow c$
  else if v is a variable node labeled by x **then**
    $\mathrm{val}(v) \leftarrow \alpha(x)$
  else if v is an addition node with children $u_1, \ldots, u_k$ **then**
    $\mathrm{val}(v) \leftarrow \mathrm{val}(u_1) + \cdots + \mathrm{val}(u_k)$
  else if v is a multiplication node with children $u_1, \ldots, u_k$ **then**
    $\mathrm{val}(v) \leftarrow \mathrm{val}(u_1) \cdots \mathrm{val}(u_k)$
  else if v is a negation node with child u **then**
    $\mathrm{val}(v) \leftarrow -\mathrm{val}(u)$
  end if
end for
return $\mathrm{val}(r)$

Because the expression is represented as a DAG rather than a tree, a shared subexpression appears as a single node and is evaluated only once. The values computed at intermediate nodes can therefore be reused wherever that subexpression occurs, which makes repeated evaluations efficient and supports the tests for linearity and vanishing described above.

Sampling points

For quick evaluation of polynomials, we will sample the random rational numbers $\frac{p}{q}$ such that $|p|, |q| < n$.

Lemma 1 *The number of destinct rational numbers we sample from is* $4(\sum_{i=1}^{n} \phi(i)) - 1$*, where* $\phi(i)$ *is the Euler's totient function. Asymptotically, this is* $\frac{12}{\pi^2} n^2 + O(n \log n)$*.*

Proof 1 *Consider the positive rational numbers* $\frac{p}{q}$ *in lowest terms (i.e.,* $\gcd(p, q) = 1$*) such that* $1 \leq p, q \leq n$*. We can partition these into two sets: those where* $p \leq q$ *and those where* $p \geq q$*. For a fixed denominator* q*, the number of numerators* $p \leq q$ *coprime to* q *is exactly Euler's totient function* $\phi(q)$*. Summing over all possible denominators* $1 \leq q \leq n$*, the number of fractions with* $p \leq q$ *is* $\sum_{i=1}^{n} \phi(i)$*. By*

symmetry, the number of fractions with $q \leq p$ is also $\sum_{i=1}^{n} \phi(i)$. The only fraction counted twice is $\frac{1}{1}$, so the total number of strictly positive rational numbers is $2\sum_{i=1}^{n} \phi(i) - 1$.

Since we can independently choose the sign of the rational number, there is an equal number of strictly negative rational numbers. Adding the single case of 0 (represented as $\frac{0}{1}$), the total number of distinct rational numbers is:

$$2\left(2\sum_{i=1}^{n}\phi(i) - 1\right) + 1 = 4\sum_{i=1}^{n}\phi(i) - 1.$$

For the asymptotic bound, it is a well-known result in number theory [2] that the summatory totient function grows as:

$$\sum_{i=1}^{n}\phi(i) = \frac{3}{\pi^2}n^2 + O(n\log n).$$

Multiplying this by 4 yields the final asymptotic bound of $\frac{12}{\pi^2}n^2 + O(n\log n)$.

If a polynomial has m free variables, then the number of points we can sample from is $|S| = (4(\sum_{i=1}^{n}\phi(i)) - 1)^m$.

Is polynomial zero

For the operation "is polynomial zero" we will use Schwartz-Zippel Lemma. We evaluate the represented expression at c sampled points of its remaining free variables. If every sampled value is zero, we treat the equation as zero. The likelihood of a polynomial $p \neq 0$ evaluating to zero at c sampled points is $(\frac{d}{|S|})^c$, where d is the degree of the polynomial [3, 4].

Is polynomial linear in a variable

The operation "is polynomial linear in a variable" can be viewed as determining if a polynomial is zero. let p be a polynomial and without the loss of generality let x_1 be a variable, we will run the zero test on the following polynomial:

$$\begin{aligned} q(u, v, w, x_2, \ldots, x_n) = &(p(w, x_2, \ldots, x_n) - p(u, x_2, \ldots, x_n))(v - u) \\ &- (p(v, x_2, \ldots, x_n) - p(u, x_2, \ldots, x_n))(w - u) \end{aligned}$$

Lemma 2 *The polynomial q is zero if and only if p is linear in x_1.*

Proof 2 *If p is linear in x_1, then there are a, b constant in x_1 such that $p(x_1, x_2, \ldots, x_n) = ax_1 + b$. Then*

$$\begin{aligned} q(u, v, w, x_2, \ldots, x_n) &= (aw + b - (au + b))(v - u) - (av + b - (au + b))(w - u) \\ &= a(w - u)(v - u) - a(v - u)(w - u) \\ &= 0. \end{aligned}$$

Conversely, assume q is zero, then for all $u, v, w, x_2, \ldots, x_n$, we have

$$\begin{aligned} 0 &= q(u, v, w, x_2, \ldots, x_n) \\ &= (p(w, x_2, \ldots, x_n) - p(u, x_2, \ldots, x_n))(v - u) \\ &\quad - (p(v, x_2, \ldots, x_n) - p(u, x_2, \ldots, x_n))(w - u) \end{aligned}$$

Thus we have

$$\frac{p(w, x_2, \ldots, x_n) - p(u, x_2, \ldots, x_n)}{w - u} = \frac{p(v, x_2, \ldots, x_n) - p(u, x_2, \ldots, x_n)}{v - u}$$

We get that there are a, b constant in x_1 such that $p(x_1, x_2, \ldots, x_n) = ax_1 + b$, and we get that p is linear in x_1.

Linearly eliminate a variable from a polynomial

For the operation "linearly eliminate a variable from a polynomial", suppose that the represented equation is linear in x_1, so it can be written as

$$ax_1 + b = 0,$$

where x is the variable to be eliminated. Since the equation is linear in x, both a and b may depend on the other variables but not on x. These coefficients can be recovered by evaluation: $b = p(0, x_2, \ldots, x_n)$ and $a = p(1, x_2, \ldots, x_n) - p(0, x_2, \ldots, x_n)$. The substitution of x will be divided into cases

- $b = 0$ we discard the equation
- $a, b \neq 0$ we discard the equation and substitute $x \mapsto -\frac{b}{a}$ without expanding the expression.
- $a = 0$ and $b \neq 0$ This is an illegal step, because the equation cannot be solved for x

Complexity Analysis

We analyze Algorithm 1 under the representation introduced in the previous section: the active system is stored as a collection of DAG-represented expressions, and solved variables are recorded in the substitution map rather than expanded eagerly into every remaining equation. This is what prevents the immediate expression swell that usually accompanies repeated substitution.

For a current elimination state (Q, Σ), let $N(Q, \Sigma)$ denote the total number of DAG nodes appearing in all active equations in Q together with all expressions stored in the substitution map Σ. We measure the cost of each operation in terms of $N(Q, \Sigma)$, and we assume throughout that arithmetic in the base field and hash-table access for memoized nodes both take constant time. We also assume that the zero test and the linearity test use a fixed number of sampled evaluation points, as described in the previous section.

Cost of the primitive operations

The basic subroutine is evaluation of a DAG-represented expression.

Lemma 3 *Let G be a DAG representing a polynomial expression. Evaluating the expression represented by G at a fixed assignment takes $O(|G|)$ time and $O(|G|)$ auxiliary space.*

Proof 3 *The evaluation algorithm processes the nodes of G in topological order. Each node is visited once, and at each visit only constant-time work is performed beyond combining the already computed values of its children. Storing one value per node gives $O(|G|)$ auxiliary space.*

This immediately yields bounds for the main decision procedures.

Lemma 4 *Let p be a represented polynomial with DAG G.*

1. *Testing whether $p \equiv 0$ by evaluating at a fixed number of sampled points takes $O(|G|)$ time.*
2. *Testing whether p is linear in a chosen variable x by evaluating at a fixed number of values of x takes $O(|G|)$ time.*
3. *If p is linear in x, recovering the coefficients a and b in $p = ax + b$ by the identities $b = p(0)$ and $a = p(1) - p(0)$ takes $O(|G|)$ time.*

Proof 4 *Each item uses only a constant number of evaluations of the DAG G. By the previous lemma, each evaluation costs $O(|G|)$ time, so the total cost remains linear in $|G|$.*

Finally, the substitution operation is linear in the size of the current represented system. When the algorithm solves an equation $p = 0$ for a variable x, it constructs the new represented expression

$$x \mapsto \frac{p(0)}{p(0) - p(1)}$$

and inserts it into Σ. This requires only the construction of a constant number of new operation nodes above already existing sub-DAGs, together with an update of the variable-to-expression map. Rewriting the remaining system to refer to this map, or traversing the active equations once to update cached metadata, costs $O(N(Q, \Sigma))$.

Cost of one elimination step

Consider one iteration of Algorithm 1. After a candidate pair (x, p) has been chosen, the algorithm performs the following work:

1. verify linearity of p in x;
2. recover the represented coefficients needed to solve for x;
3. insert the solved form into the substitution map;
4. identify and remove equations that have become zero.

Each item is completed by a constant number of traversals or evaluations of the current representation, so one elimination step costs $O(N(Q, \Sigma))$ time. In particular, the cost is linear in the DAG size of the current state, not in the size of a fully expanded polynomial system.

If the elimination process performs d successful elimination steps and the represented system sizes along the run are

$$N_0, N_1, \ldots, N_d,$$

then the total running time of the elimination phase is

$$O\left(\sum_{i=0}^{d-1} N_i\right).$$

Thus the runtime is governed by how the represented size grows from one elimination step to the next.

Growth of the represented system

The advantage of the substitution-map representation is that an elimination step does not duplicate an entire solved expression at every occurrence of the eliminated variable. Instead, the solved expression is stored once, and future uses refer to the same DAG. Consequently, a single step can increase the represented size only by the new nodes needed to encode the solved form.

This rules out the immediate combinatorial explosion caused by eager expansion, but it does not completely prevent growth. If each elimination step introduces a new solved expression whose numerator and denominator depend on several previously created expressions, then the represented size may still increase by a constant factor from one depth level of elimination to the next. Therefore the worst-case growth of the represented system is bounded by an exponential function of the elimination depth.

Theorem 2 *There exists a constant $c > 1$ such that after d successful elimination steps,*

$$N_d \leq c^d N_0,$$

and therefore the total running time of the elimination phase is bounded by

$$O\big(N_0 c^d\big)\,.$$

Proof 5 *At each elimination step, the algorithm adds only a constant number of new operation nodes on top of already existing DAGs and updates the substitution map. Hence the represented size after one step is at most a constant multiple of the represented size before that step:*

$$N_{i+1} \leq cN_i$$

for some implementation-dependent constant c. Iterating this inequality gives $N_d \leq c^d N_0$. Substituting this bound into

$$\sum_{i=0}^{d-1} N_i$$

yields a geometric series, and therefore the total running time is $O(N_0 c^d)$.

In summary, vanishing tests, linearity tests, coefficient recovery, zero-equation detection, and substitution-map updates are all linear in the current DAG size of the represented system. The only source of super-linear overall behavior is the growth of that represented system across many elimination steps, and under the DAG-based representation this growth is at worst exponential in the elimination depth rather than in the fully expanded polynomial size.

Handling systems without linearities

the polynomial order we will use to interreduce, and make polynomials monic, should maximize the number of monomials with that are linear in one of any of the variables. thus we should use the grevlex order.

Experiments

Experimental Setup

The implementation of our was written in Rust and is available at `https://github.com/IlayMenahem/soothing`. We ran our experiments on ...

Results

Conclusion and Future Work

References

[1] Stepan Yu Gatilov. Using low-rank approximation of the jacobian matrix in the newton–raphson method to solve certain singular equations. *Journal of Computational and Applied Mathematics*, 272:8–24, 2014.

[2] Godfrey Harold Hardy and Edward Maitland Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6th edition, 2008.

[3] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980.

[4] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation: EUROSAM'79, An International Symposium on Symbolic and Algebraic Computation Marseille, France, June 1979 Proceedings*, pages 216–226. Springer, 1979.

Appendix

Simplifying expressions

To simplify an expression, we can use the following local simplifications. Some are always semantics-preserving, while others are valid only under additional side conditions.

Always safe local simplifications.

- $x_1 + \ldots + 0 + \ldots + x_n \mapsto x_1 + \ldots + x_n$
- $x_1 + \ldots + x + \ldots + (-x) + \ldots + x_n \mapsto x_1 + \ldots + x_n$
- $-(-x) \mapsto x$
- $x_1 \cdot \ldots \cdot 1 \cdot \ldots \cdot x_n \mapsto x_1 \cdot \ldots \cdot x_n$
- $x_1 \cdot \ldots \cdot 0 \cdot \ldots \cdot x_n \mapsto 0$
- $((-1)^{n_1} \cdot x_1) \cdot \ldots \cdot ((-1)^{n_k} \cdot x_k) \mapsto (-1)^{n_1 + \ldots + n_k} \cdot (x_1 \cdot \ldots \cdot x_k)$ when n_i are integers
- $\frac{x}{1} \mapsto x$
- $\frac{-x}{y} \mapsto -\frac{x}{y}$
- $\frac{x}{-y} \mapsto -\frac{x}{y}$
- $\frac{-x}{-y} \mapsto \frac{x}{y}$
- $\frac{x_1}{y_1} \cdot \ldots \cdot \frac{x_n}{y_n} \mapsto \frac{x_1 \cdot \ldots \cdot x_n}{y_1 \cdot \ldots \cdot y_n}$
- $\frac{\frac{x}{y}}{\frac{z}{w}} \mapsto \frac{x \cdot w}{y \cdot z}$
- $x^1 \mapsto x$
- $1^n \mapsto 1$
- constant folding for addition, multiplication, division, and powers whenever the result is defined, if the result is undefined give an error.
- flatten nested additions and multiplications: $x_1 + \ldots + (y_1 + \ldots + y_m) + \ldots + x_n \mapsto x_1 + \ldots + y_1 + \ldots + y_m + \ldots + x_n$

- combine affine terms: $a_1 \cdot x + \ldots + a_k \cdot x \mapsto (a_1 + \ldots + a_k) \cdot x$ when a_i are constants

Local simplifications requiring side conditions.

- $\frac{x}{x} \mapsto 1$ if $x \neq 0$
- $\frac{0}{y} \mapsto 0$ if $y \neq 0$
- $\frac{x \cdot y}{x \cdot z} \mapsto \frac{y}{z}$ if $x \neq 0$ and $z \neq 0$
- $\frac{x_1 \cdot \ldots \cdot x_i \cdot \ldots \cdot x_n}{y_1 \cdot \ldots \cdot x_i \cdot \ldots \cdot y_m} \mapsto \frac{x_1 \cdot \ldots \cdot x_{i-1} \cdot x_{i+1} \cdot \ldots \cdot x_n}{y_1 \cdot \ldots \cdot y_{i-1} \cdot y_{i+1} \cdot \ldots \cdot y_m}$ if $x_i \neq 0$ and $y_i \neq 0$
- $x^0 \mapsto 1$ if $x \neq 0$, or if 0^0 is defined by convention
- $0^n \mapsto 0$ for positive integers n

Local simplifications algorithm. To apply all the local simplifications efficiently in a single pass over the expression DAG, we use a bottom-up traversal with memoization. This ensures that every unique subexpression is simplified exactly once, and that parent nodes are simplified only after their children have been fully simplified. The ConstructAndSimplify function acts as a "smart constructor" that applies the local simplification rules to a node given its already-simplified children.

When multiple simplification rules could apply within the smart constructor, they are evaluated in the following preference order to ensure termination and maximal simplification:

1. **Constant folding:** If all operands are constants, immediately compute the result.
2. **Flattening:** For associative operations (addition and multiplication), flatten nested identical operations. This exposes all operands to the subsequent steps.
3. **Annihilators and Identities:** Apply rules involving 0 and 1 (e.g., $x \cdot 0 \mapsto 0$, $x + 0 \mapsto x$, $x^1 \mapsto x$).
4. **Sign and Fraction Normalization:** Push negations down or pull them up consistently, and simplify fractions (e.g., $-(-x) \mapsto x$, $\frac{-x}{-y} \mapsto \frac{x}{y}$).
5. **Combining like terms:** Group constants together (partial constant folding among siblings) and combine affine terms (e.g., $a \cdot x + b \cdot x \mapsto (a + b) \cdot x$).

Algorithm 3 Bottom-up DAG simplification

Require: DAG node v, memoization cache C
Ensure: A fully simplified DAG node

function SIMPLIFY(v, C)
  if $v \in C$ **then**
    return $C[v]$
  end if
  if v is a variable or constant **then**
    $C[v] \leftarrow v$
    return v
  end if
  Let $u_1, \ldots, u_k$ be the children of v
  for $i = 1 \ldots k$ **do**
    $u'_i \leftarrow$ SIMPLIFY(u_i, C)
  end for
  $v' \leftarrow$ CONSTRUCTANDSIMPLIFY(operation of $v, u'_1, \ldots, u'_k$)
  $C[v] \leftarrow v'$
  return v'
end function

DFS for linear elimination order

In this setting, DFS means that we choose one available linear elimination, apply it, and continue recursively on the reduced system until no further linear elimination is possible. Only then do we backtrack and try a different elimination choice from an earlier step.

Algorithm 4 DFS search for a linear elimination order

Require: Polynomial system P
Ensure: A reduced system Q together with a substitution map Σ

function DFSLinearElimination(P, Σ)
  $L \leftarrow \{(p, x) \mid p \in P,\ p \text{ is linear in } x\}$
  if $L = \emptyset$ or $P = \emptyset$ **then**
    return (P, Σ)
  end if
  $B \leftarrow (P, \Sigma)$
  for all $(p, x) \in L$ **do**
    solve $p = 0$ for x to obtain $x \leftarrow f(\hat{x})$
    $P' \leftarrow P$ with $x \leftarrow f(\hat{x})$ substituted in every equation
    remove zero equations from P'
    $\Sigma' \leftarrow \Sigma \cup \{x \mapsto f(\hat{x})\}$
    $R \leftarrow$ DFSLinearElimination(P', Σ')
    if $R = (\emptyset, \Sigma'')$ for some Σ'' **then**
      return R
    end if
    if R is better than B **then**
      $B \leftarrow R$
    end if
  end for
  return B
end function

Here "better" means that the resulting system has fewer remaining equations, and if the number of remaining equations is the same, then we prefer the system with more free variables.